UNIVERSITY OF OXFORD

AI Engineering with MCP

Build Production-Ready AI Applications with Model Context Protocol

Hands-On Demos

Enterprise Patterns

YOUR SPEAKER

Vikkas Arun Pareek

20+ years in IT — currently at Barclays

Author of 5 books — fiction and children's

Numerologist · Reiki Grand Master · Tarot Reader · Vastu Consultant

GenAI Instructor — Cohort 5 · 50+ participants trained

Post-course: 1 app on Google Play · 5 SaaS products in production · more in pipeline

Connect on LinkedIn linkedin.com/in/vikkas-arun-pareek-20790918/

TODAY'S SESSION

What We'll Cover

14:30 – 14:35	Introduction & Welcome	5 min
14:35 – 14:40	Live Demo — What can MCP do!	5 min
14:40 – 14:50	Revisit the Basics	10 min
14:50 – 15:00	API Demo — Why AI Needs More	10 min
15:00 – 15:15	What is MCP + Key Components	15 min
15:15 – 15:30	Client & Server Code Walkthrough	15 min
15:30 – 16:00	Use Case — IT Support (live demo)	30 min
16:00 – 16:20	Use Case — ModelMyWealth (live demo)	20 min
16:20 – 16:30	Q&A and Thank You	10 min

UNIVERSITY OF OXFORD

Live Demo - MCP Dungeon Game

1. An AI agent navigating a 4×4 dungeon — no script, no hard-coded logic
2. Every action is a tool call: move() · attack() · pick_up() · inspect_room()
3. Agent reads tool descriptions at runtime and decides what to call
4. Goal: find the ancient key · defeat enemies · escape through the iron door

Live application demo →

Same pattern as IT Support · Same pattern as ModelMyWealth · Different domain, identical architecture

UNIVERSITY OF OXFORD

Four things you already know

1. Chatbot (LangChain)

An LLM answering questions.
LangChain chains prompt → model → output.

2. RAG

Structured: SQL queries on a database.
Unstructured: semantic search over docs.

3. AI Agents

LangGraph state machine.
Agent decides what to do next.

4. APIs

HTTP endpoints apps call each other with.
Built for humans, not for AI.

MCP connects all four. It's the missing standard that makes them talk to AI.

UNIVERSITY OF OXFORD

API Demo

Developer knows the endpoint

You hard-code GET /tickets/42. The URL was written by a human who read the docs.

API returns raw data

The response is a JSON blob. No description. No context. The AI has no idea what to do with it.

Every API is different

GET for reads, POST for writes. 404 = not found, 500 = crashed. The AI must be taught each rule.

Problem: AI can't read a spec sheet and hard-code URLs at runtime. It needs something different.

UNIVERSITY OF OXFORD

Three problems. One solution.

1. Discovery
APIs assume you know what endpoints exist. AI must find them at runtime.

2. Context
APIs return raw data. AI needs descriptions and schemas built in.

3. Standard
Every API is different. AI needs one consistent protocol.

MCP is a standard protocol for AI to discover and use external tools.

UNIVERSITY OF OXFORD

MCP is a standard protocol for AI to discover and use external tools.

Like USB-C — one connector, any device, any manufacturer. It just works.

One protocol. Any AI client. Any tool server. They just connect.

UNIVERSITY OF OXFORD

Key components.

MCP Client

The AI — discovers & calls tools

Claude Desktop, LangGraph agent, VS Code.
Asks: "What tools do you have?" then decides.

MCP Server

Your tools & data

Exposes tools (actions) and resources (data).
No AI inside — it just runs your code.

Tool

An action the AI can take

Name + description + Zod schema + handler.
Description = AI's instruction manual.

Resource

Data the AI can read

Read-only URI endpoints (tickets://catalog).
Like a GET — no side effects.

Transport

How they communicate

JSON-RPC 2.0 over stdio (local) or HTTP-SSE.
Vendor-neutral — any client, any server.

listTools()

The discovery mechanism

Called at startup. Server returns all schemas.
The AI reads descriptions — decides at runtime.

UNIVERSITY OF OXFORD

The architecture diagram.

MCPArchitecture.html

Open the interactive diagram now.

Click any node — Chat Interface → Orchestrator → Agents → Protocol → MCP Servers → Data Layer

1. User → Chat UI → FastAPI Backend
2. LangGraph Orchestrator (multi-agent)
3. MCP Protocol (JSON-RPC)
4. MCP Servers: RAG · Data · Comms · Web
5. Data: Vector DB · PostgreSQL · Slack · Internet

UNIVERSITY OF OXFORD

MCP client — four steps.

```
const path = require("node:path");
const { Client } = require("@modelcontextprotocol/sdk/client/index.js");
const { StdioClientTransport } = require("@modelcontextprotocol/sdk/client/stdio.js");
const client = new Client({ name: "c", version: "1" }, { capabilities: {} });
await client.connect(
 new StdioClientTransport({
   command: process.execPath,
   args: [path.join(__dirname, "server.js")],
   cwd: __dirname,
   env: process.env,
 })
);
const { tools } = await client.listTools();
const out = await client.callTool({ name: "ping", arguments: {} });
console.log(tools.map((t) => t.name), out);
await client.close();
```

UNIVERSITY OF OXFORD

MCP server

```
const { McpServer } = require("@modelcontextprotocol/sdk/server/mcp.js");
const { StdioServerTransport } =
require("@modelcontextprotocol/sdk/server/stdio.js");
const { z } = require("zod");
const server = new McpServer(
 { name: "s", version: "1" },
 { capabilities: { tools: {} } }
);
server.tool("ping", "Returns pong.", {}, async () => ({
 content: [{ type: "text", text: "pong" }],
}));
server.connect(new StdioServerTransport());
```

UNIVERSITY OF OXFORD

Demo - Oxford University Support Portal

`Open: ITSupportArchitecture.html · Code: backend/mcp_server/server.ts`

1. **React UI + FastAPI Backend**
   Employees chat in the browser. Responses stream in real time via SSE.
2. **LangGraph Orchestrator**
   Routes each message through Triage → RAG → (optionally) Ticket → Response agents.
3. **RAG Agent**
   Searches the IT knowledge base (Chroma vector store). If confident, answers directly.
4. **Ticket Agent + MCP**
   When RAG can't resolve the issue, the agent calls create_ticket() via MCP. Human confirmed first.
5. **6 MCP Tools**
   create_ticket · get_ticket · update_status · check_system · reset_password · search_known_issues

UNIVERSITY OF OXFORD

Demo - Model My Wealth

1. An AI assistant that uses MCP tools to talk to real ModelMyWealth data
2. MCP server calls your REST API over HTTP with a Bearer JWT
3. Agent answers: "What if I invested £10k in index funds for 20 years?"
4. Same MCP pattern — different domain, identical architecture

Live application demo →

Thank You.

Questions?

GitHub - https://github.com/vapmail16/oxford_mcp